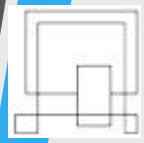
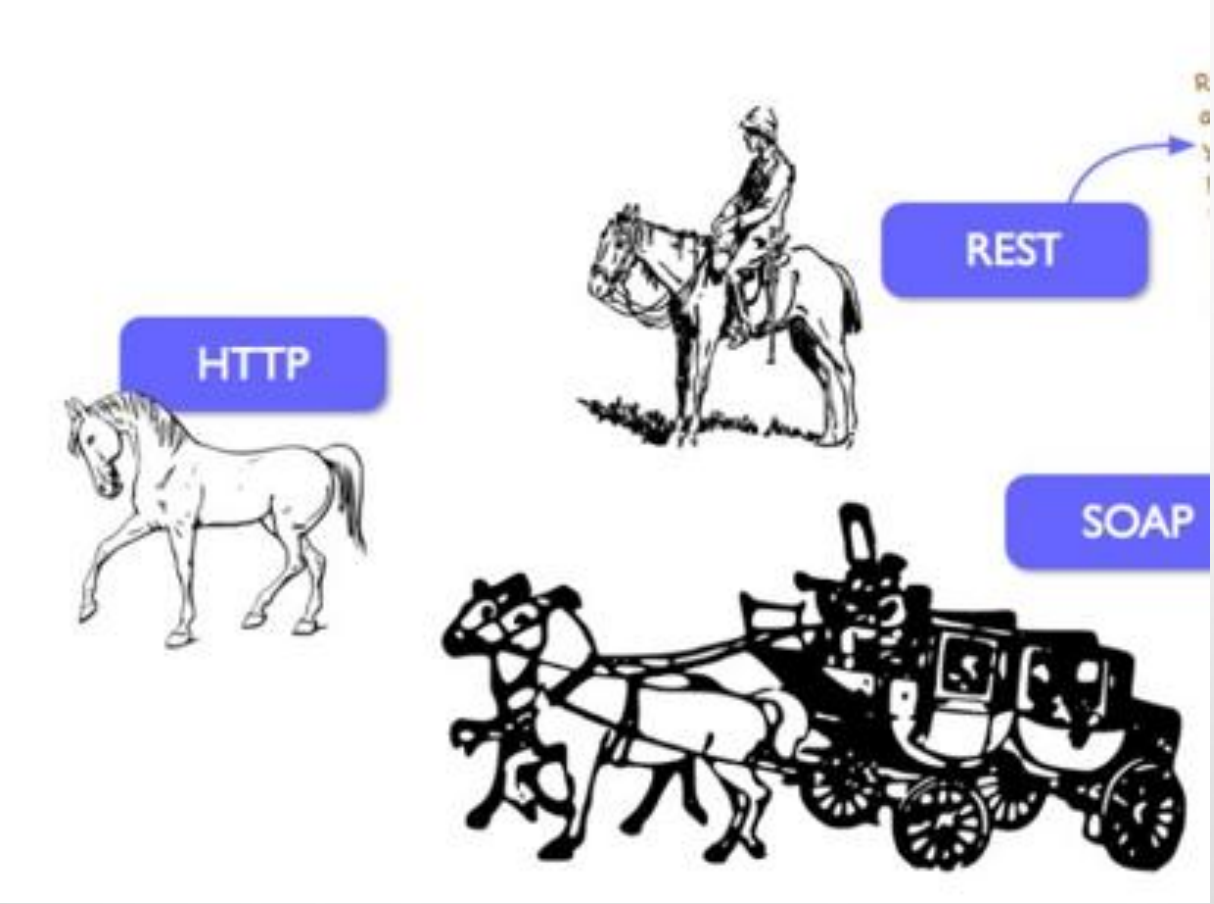
VIA University College



Programming 3

Autumn 2026

Web Services



Learning Objectives

- By the end of this session, you should be able to:
  - ✓ explain web services
  - ✓ explain REST (JAX-RS)
  - ✓ implement REST services using Spring Boot

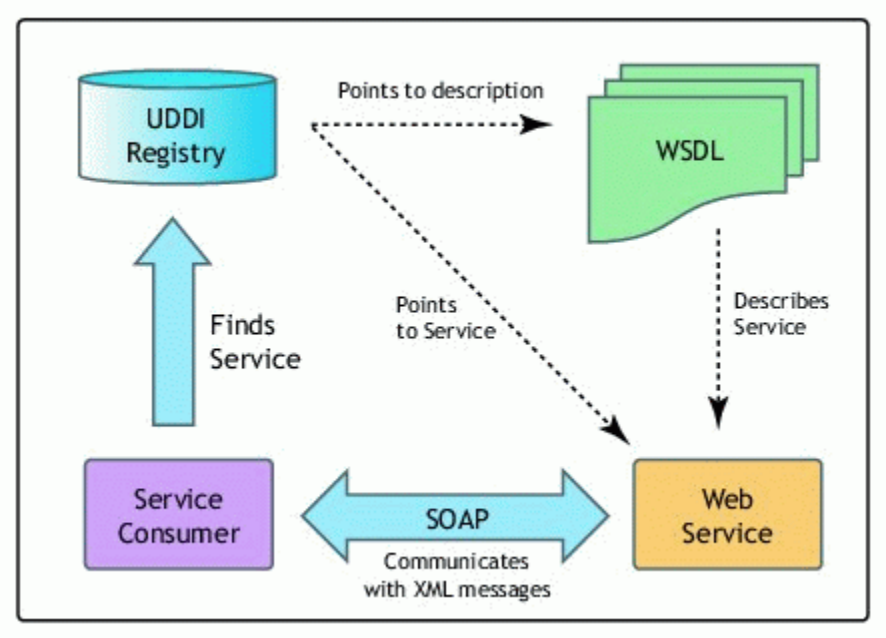
Web Services I

- provide a service interface enabling clients to interact with servers in a more general way than web browsers do
- Clients make requests and replies are formatted in XML(or JSON) and usually transmitted over HTTP
- Users require a secure means for creating, storing and modifying documents and exchanging them over the Internet
- provide an infrastructure for maintaining a richer and more structured form of interoperability between clients and servers

Web Services II

- provide a basis whereby a client program in one organization may interact with a server in another organization without human supervision
- External data representation and marshalling of messages exchanged between clients and web services is done in XML (SOAP).
- The SOAP (Simple Object Access Protocol) specifies the rules for using XML to package messages, for example to support a request-reply protocol

Web Services - architecture



*architectural representation of the Web Services model

(https://support.novell.com/techcenter/articles/dnd20030304.html)

Web Services <-> Distributed Systems

- One of the dominant paradigms for programming distributed systems
- Enables business to business integration. (Suppose one organization uses CORBA and another uses .NET) No problem!
- Enables service-oriented architecture (SOA).
- May exist internally to an organization or externally (in the cloud).

How did Web Services emerge?

- Web Services began life when Bill Gates introduced BizTalk in 1999.
- BizTalk later renamed .NET.
- The idea: "to provide the technologies to allow software in different places, written in different languages and resident on different platforms to connect and interoperate." From "Programming the World Wide" by Sebesta
- Two approaches
  - SOAP Based (WS-*) Web Services
  - REST style web services

JAVA Web Services API

- Web services are client and server applications that communicate over HTTP to implement RPCs
  - are language- and platform-independent
  - Data is transmitted in a standardized XML format (*)
- RESTful Web Services
  - Java API for RESTful Web Services (JAX-RS)
  - No requirement of XML messages or WSDL service
- Additional information: https://www.oracle.com/technetwork/java/javaee/overview/index.html

JAX-RS

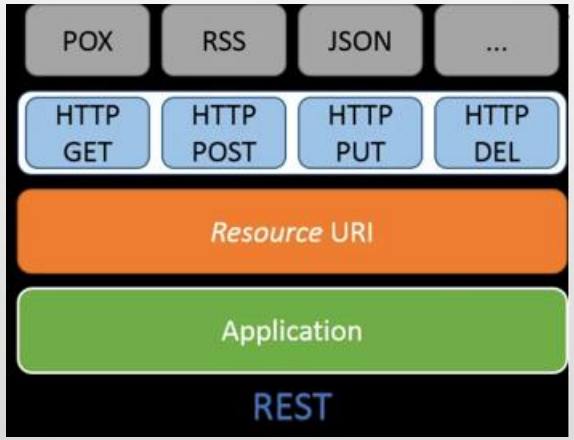
- Representational State Transfer (REST) is an architectural style of client-server applications centered around the transfer of resources through requests and responses using Uniform Resource Identifiers (URIs)
  - Communication is over HTTP
  - A REST resource might be XML, HTML, plain text, PDF, JPEG, JSON, or other types
  - This resource can be retrieved, updated, or deleted based on permissions
  - REST uses HTTP, which is stateless, but state can be maintained with cookies, URI rewriting, and hidden form fields
  - https://www.oracle.com/technetwork/java/javaee/overview/index.html

Representational State Transfer (REST)

- A style of software architecture for distributed hypermedia systems such as the World Wide Web.
- Introduced in the doctoral dissertation of Roy Fielding
  - One of the principal authors of the HTTP specification.
- A collection of network architecture principles which outline how resources are defined and addressed

REST

- JAX-RS is used for web applications using the REST architectural pattern
  - Stateless operations (can withstand server restart)
  - Scalability
  - Architectural simplicity (use off the shelf components)
  - Resources can be accessed using a variety of formats



Hypertext Transfer Protocol (HTTP)

- A communication protocol
- Allows retrieving inter-linked text documents (hypertext)
  - World Wide Web.
- HTTP Verbs
  - HEAD
  - **GET**
  - **POST**
  - PUT
  - DELETE
  - PATCH
  - TRACE
  - OPTIONS
  - CONNECT



REST and HTTP

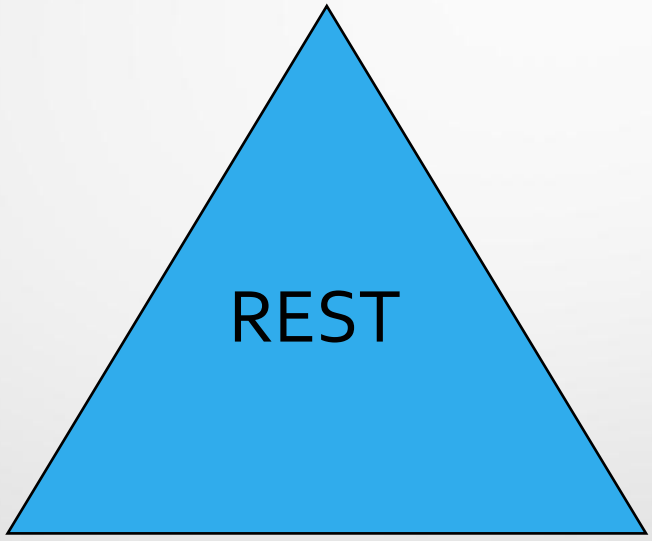
- REST was to capture the characteristics of the Web
  - URI Addressable resources
  - HTTP Protocol
  - Make a Request – Receive Response – Display Response
- Exploits the use of the HTTP protocol beyond HTTP POST and HTTP GET
  - HTTP PUT, HTTP DELETE

REST - not a Standard

- REST is not a standard
  - JSR 311: JAX-RS: The Java™ API for RESTful Web Services
- But it uses several standards:
  - HTTP
  - URL
  - XML/JSON/HTML/GIF/JPEG/etc (Resource Representations)
  - text/xml, text/html, image/gif, image/jpeg, etc (Resource Types, MIME Types)

Main Concepts

Nouns (Resources)
unconstrained
i.e., http://localhost:8080/api/v1/animal/201



Verbs
constrained
i.e., GET

Representations
constrained
i.e., JSON

Resources

- The key abstraction of information in REST is a resource.
- A resource is a conceptual mapping to a set of entities
  - Any information that can be named can be a resource: a document or image, a temporal service (e.g. "today's menu in VIA's canteen"), a collection of other resources, a non-virtual object (e.g. an animal), and so on
- Represented with a global identifier (URI in HTTP)
  - http://www.exoticslaughter.com/animal/202

Naming Resources

- REST uses URI to identify resources
  - http://localhost/api/animal/
  - http://localhost/api/animal/201
  - http://localhost/api/animal/201/product
- As you traverse the path from more generic to more specific, you are navigating the data

Verbs

- Represent the actions to be performed on resources

- HTTP GET
- HTTP POST
- HTTP PUT (or PATCH)
- HTTP DELETE
- etc

HTTP Method	Description
GET	Provides a read only access to a resource.
POST	Used to create a new resource.
DELETE	Used to remove a resource.
PUT	Used to update an existing resource or create a new resource.

HTTP GET

- How clients ask for the information they seek.
- Issuing a GET request transfers the data from the server to the client in some representation
- GET http://localhost/api/animals
  - Retrieve all animals
- GET http://localhost/api/animals/201
  - Retrieve animal identified with id 201

HTTP PUT, HTTP POST

- HTTP POST creates a resource
- HTTP PUT updates a resource

- POST http://localhost/api/animals/
  - Content: {name, origin, …}
  - Creates/registers a new animal with given properties

- PUT http://localhost/api/animals/211
  - Content: {animal_id, name, origin, …}
  - Updates animal identified by 211 with submitted properties

HTTP DELETE

- Removes the resource identified by the URI
- DELETE http://localhost/api/animals/211
  - Delete animal identified by animal_id 211

Representations

- How data is represented or returned to the client for presentation.
- Two main formats:
  - JavaScript Object Notation (JSON)
  - XML
- It is common to have multiple representations of the same data

Representations

- XML

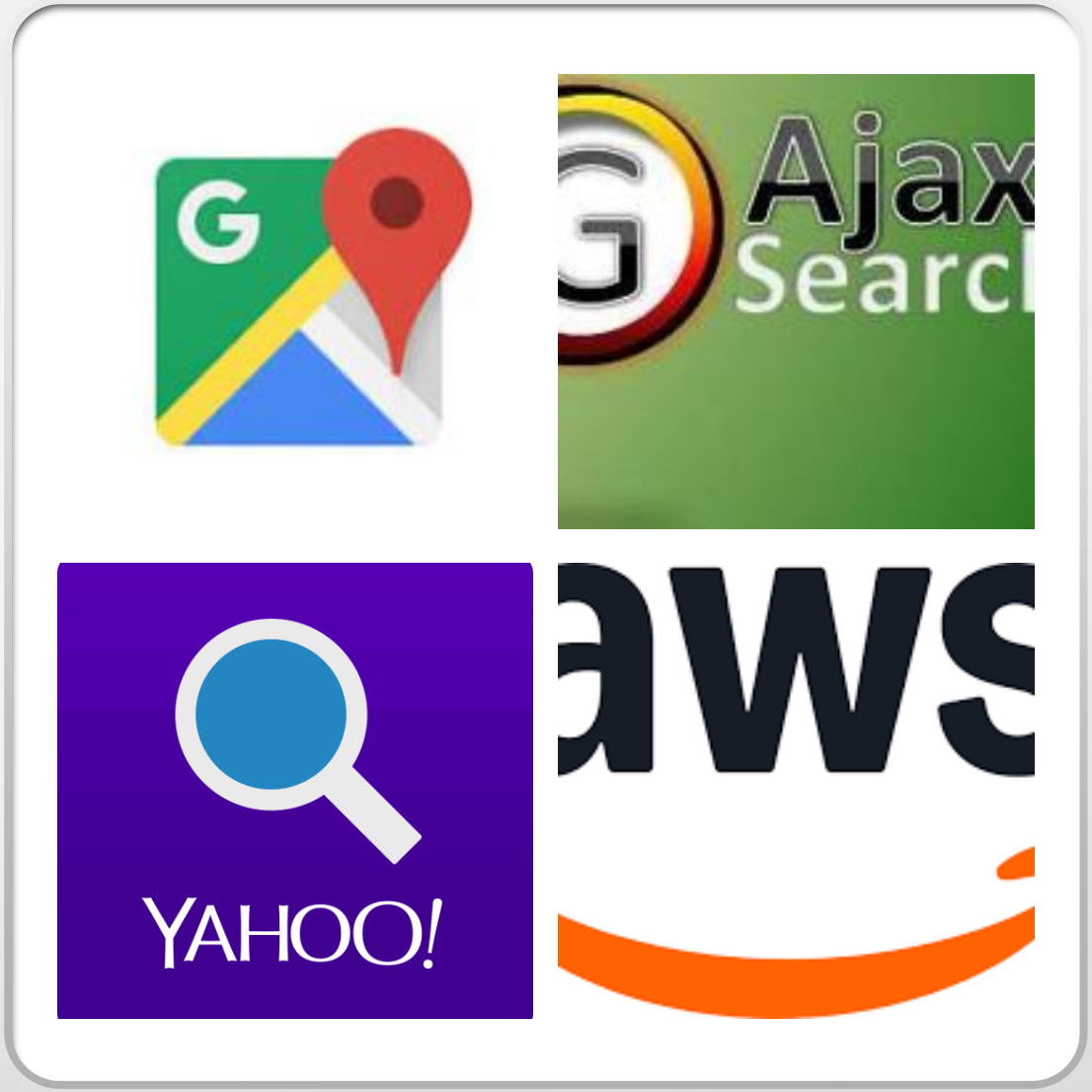
```
<ANIMAL>
    <ID>211</ID>
    <NAME>Chicken</NAME>
</ANIMAL>
```

- JSON

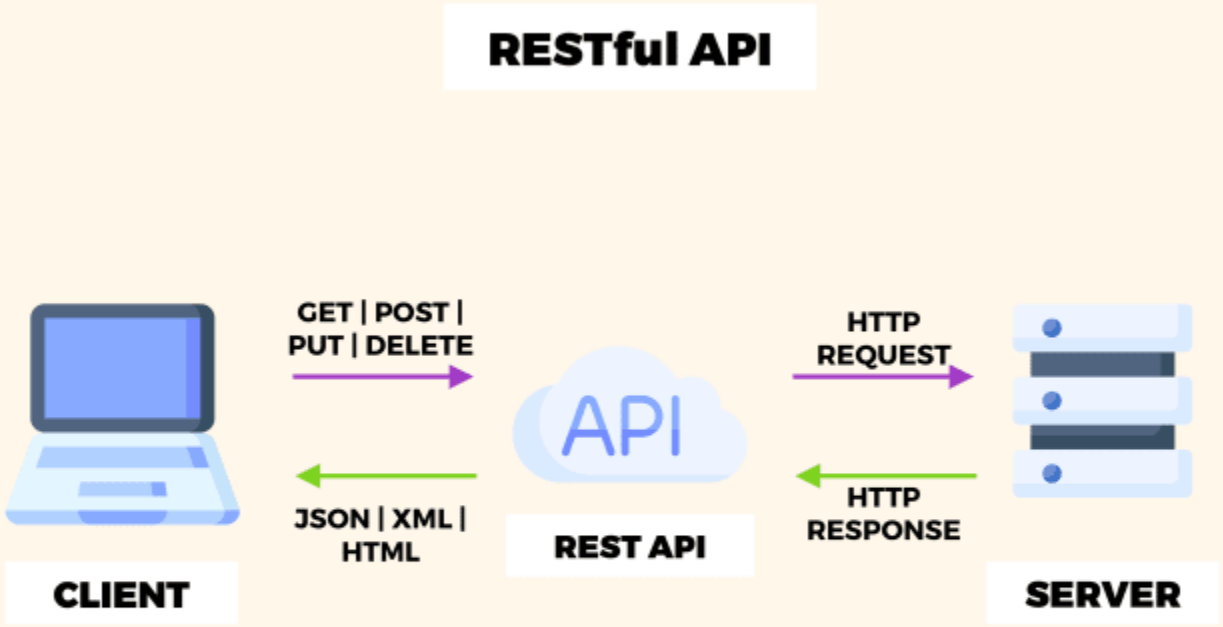
```
{animal
    {id: 211}
    {name: Chicken}
}
```

Real Life Examples

- Google Maps
- Google AJAX Search API
- Yahoo Search API
- Amazon WebServices



REST APIs



https://velog.io/@dhnjun9/REST-REST-API-RESTful

Designing a REST api

How should you think?

- Make it as easy as possible for the other programmers.

Facebook:

- Always return 200 OK.
- `GET /v2.7/{user-id}`
- `GET /v2.7/{post-id}`
- `GET /v2.7/{user-id}/friends`
- `GET /v2.7/{object-id}/likes`

Live Coding/Hands-on/Demo

